

Programarea Funcțională în Java

Introducere

Începând cu versiunea Java 8, limbajul Java a introdus suport extins pentru programarea funcțională. Acest stil de programare permite scrierea unui cod mai concis, mai lizibil și mai ușor de întreținut, utilizând concepte precum expresii lambda, interfețe funcționale și referințe către metode.

Programarea funcțională se bazează pe ideea tratării funcțiilor ca obiecte de primă clasă, ceea ce înseamnă că acestea pot fi transmise ca parametri, returnate din metode și stocate în variabile.

Elementele de Programare Funcțională în Java

Programarea funcțională este un paradigm de programare în care accentul cade pe:

- utilizarea funcțiilor;
- evitarea modificării stării obiectelor;
- folosirea expresiilor în locul instrucțiunilor complexe;
- prelucrarea colecțiilor prin operații declarative.

În Java, programarea funcțională este implementată prin:

- expresii lambda;
- interfețe funcționale;
- Stream API;
- referințe către metode;
- funcții predefinite din pachetul `java.util.function`.

Avantajele programării funcționale

- Cod mai scurt și mai clar
- Ușurință în procesarea colecțiilor
- Cod mai ușor de întreținut
- Posibilitatea procesării paralele

Interfețe funcționale

O interfață funcțională este o interfață care conține o singură metodă abstractă.

Exemplu:

```
@FunctionalInterface
interface Calcul {
    int operatie(int a, int b);
}
```

Această interfață poate fi implementată folosind expresii lambda.

Expresii Lambda

Definiție

Expresiile lambda reprezintă funcții anonime care permit transmiterea comportamentului ca parametru.

Sintaxa generală:

(parametri) -> expresie

sau

```
(parametri) -> {
    instructiuni
}
```

Componentele unei expresii lambda

Lista de parametri

(x, y)

Operatorul lambda

->

Corpul expresiei

x + y

Exemplu:

```
Calcul suma = (a, b) -> a + b;  
  
System.out.println(suma.operatie(3, 5));  
  
//Rezultat:  
  
8
```

Avantajele expresiilor lambda

- reduc cantitatea de cod;
- cresc lizibilitatea;
- permit programare declarativă;
- facilitează utilizarea Stream API.

Limitări

- pot reduce lizibilitatea dacă sunt foarte complexe;
- variabilele utilizate trebuie să fie final sau effectively final.

Exemplu:

```
int x = 10;  
  
Runnable r = () -> System.out.println(x);
```

Variabila x nu trebuie modificată ulterior.

Descriptori Funcționali

Definiție

Descriptorul funcțional reprezintă semnătura metodei abstracte dintr-o interfață funcțională.

Practic, acesta descrie:

- tipul parametrilor;
- tipul valorii returnate.

Exemplu

Interfață funcțională:

```
@FunctionalInterface
interface Operatie {
    int calculeaza(int a, int b);
}
```

Descriptorul funcțional este:

`(int, int) -> int`

Adică:

- primește doi parametri de tip `int`;
- returnează un `int`.

Rolul descriptorilor

Descriptorii permit compilatorului Java să determine dacă o expresie lambda este compatibilă cu o interfață funcțională.

Exemplu:

```
Operatie adunare = (a, b) -> a + b;
```

Compilatorul verifică dacă lambda respectă descriptorul:

`(int, int) -> int`

Compatibilitate

Exemplu corect:

```
Operatie inmultire = (a, b) -> a * b;
```

Exemplu incorect:

```
Operatie test = (a, b) -> "text";
```

Eroarea apare deoarece metoda trebuie să returneze `int`, nu `String`.

Predicate, Consumer, Function și Supplier

Aceste interfețe sunt definite în pachetul:

`java.util.function`

Ele sunt utilizate frecvent în programarea funcțională.

Predicate

Definiție

Predicate<T> reprezintă o funcție care primește un parametru și returnează o valoare booleană.

Descriptor funcțional:

(T) -> boolean

Metoda principală

boolean test(T t)

Exemplu

```
Predicate<Integer> estePar = x -> x % 2 == 0;

System.out.println(estePar.test(4));
System.out.println(estePar.test(7));

//Rezultat:
true
false
```

Utilitate

Predicatele sunt folosite pentru:

- filtrare
- validare
- condiții logice

Consumer

Definiție

Consumer<T> primește un parametru și nu returnează nimic.

Descriptor funcțional:

(T) -> void

Metoda principală

void accept(T t)

Exemplu

```
Consumer<String> afisare = text -> System.out.println(text);  
afisare.accept("Java Functional");
```

Utilitate

Consumer este utilizat pentru:

- afișare;
- salvare date;
- modificări asupra obiectelor.

Exemplu cu listă

```
List<String> nume = Arrays.asList("Ana", "Ion", "Maria");  
nume.forEach(x -> System.out.println(x));
```

Function

Definiție

Function<T, R> primește un parametru și returnează o valoare.

Descriptor funcțional:

(T) -> R

Metoda principală

R apply(T t)

Exemplu

```
Function<String, Integer> lungime = text -> text.length();  
System.out.println(lungime.apply("Java"));  
//Rezultat:  
4
```

Utilitate

Function este folosit pentru:

- transformări;
- conversii;
- mapări.

Supplier

Definiție

Supplier<T> nu primește parametri și returnează o valoare.

Descriptor funcțional:

() -> T

Metoda principală

T get()

Exemplu

```
Supplier<Double> random = () -> Math.random();  
System.out.println(random.get());
```

Utilitate

Supplier este utilizat pentru:

- generare de valori;
- lazy initialization;
- creare obiecte.

Exemplu

```
Supplier<Date> dataCurenta = () -> new Date();  
System.out.println(dataCurenta.get());
```

Referințe către Metode

Definiție

Referințele către metode reprezintă o formă prescurtată a expresiilor lambda.

Sintaxa:

Clasa::metoda

Tipuri de referințe

Referință către metodă statică

Integer::parseInt

Exemplu:

```
Function<String, Integer> conversie = Integer::parseInt;  
System.out.println(conversie.apply("25"));
```

Referință către metodă de instanță

System.out::println

Exemplu:

```
List<String> lista = Arrays.asList("A", "B", "C");  
lista.forEach(System.out::println);
```

Referință către metodă a unui obiect arbitrar

String::toUpperCase

Exemplu:

```
List<String> lista = Arrays.asList("ana", "maria");  
lista.stream()  
    .map(String::toUpperCase)  
    .forEach(System.out::println);
```

Referință către constructor

Sintaxă:

Clasa::new

Exemplu:

```
Supplier<List<String>> lista = ArrayList::new;
```

Avantaje

- cod mai scurt;
- lizibilitate crescută;
- reutilizare mai bună a metodelor existente.

Diferența dintre lambda și referință către metodă

Lambda:

```
x -> System.out.println(x)
```

Referință:

```
System.out::println
```

Metoda forEach

Definiție

Metoda `forEach()` permite iterarea prin elementele unei colecții folosind programare funcțională.

Aceasta primește un obiect de tip `Consumer`.

Sintaxă

```
forEach(Consumer<T> action)
```

Exemplu simplu

```
List<String> lista = Arrays.asList("Ana", "Ion", "Maria");  
  
lista.forEach(x -> System.out.println(x));
```

Exemplu cu referință către metodă

```
lista.forEach(System.out::println);
```

Utilitate

Metoda `forEach()` este utilizată pentru:

- parcurgerea colecțiilor;
- afișarea elementelor;
- aplicarea unei operații asupra fiecărui element.

Exemplu cu modificare

```
List<Integer> numere = Arrays.asList(1,2,3,4);  
  
numere.forEach(x -> System.out.println(x * 2));  
  
//Rezultat:  
2  
4  
6  
8
```

Exercitii

1. Se dă o listă de numere întregi. Să se afișeze doar numerele pare mai mici ca 17 utilizând:
 - interfața `Predicate`;
 - expresii `lambda`;
 - metoda `forEach()`.
2. Creați un obiect `supplier` care sa genereze numere aleatoare pare de la 1 la 100.
3. Se dă o listă de nume. Transformati toate numele în majuscule si eliminati-le pe cele mai scurte de 3 litere. Afișati lista ramasa utilizand `forEach`.
4. Creați o clasă `Student` care conține:
 - `nume`;
 - `medie`.

Introduceți mai mulți studenți într-o listă și sortați lista:

- alfabetic după nume;
- descrescător după medie.

5. Se definește clasa Produs cu:

- nume;
- categorie;
- preț;
- stoc.

Creați minimum 10 produse și realizați următoarele:

- afișați produsele care au stoc mai mic de 5;
- calculați valoarea totală a stocului magazinului;
- sortați produsele după categorie și apoi după preț;
- aplicați o reducere de 10% produselor din categoria „Electronice”;
- afișați doar numele produselor;
- creați un Supplier<Produs> care generează produse noi cu valori random;
- afișați toate produsele utilizând forEach() și referințe către metode.